MIC: A High-Throughput Lossless Codec for Medical Images

Kuldeep Singh*

Abstract

We present MIC (Medical Image Codec), a lossless compression codec optimized for 16-bit medical images stored in the DICOM format. MIC chains three lightweight stages—spatial delta prediction, run-length encoding (RLE), and finite state entropy (FSE) coding based on asymmetric numeral systems (ANS)—to achieve compression ratios of 1.7×–8.6× on greyscale DICOM images and 3–5× on RGB whole slide imaging (WSI) tissue tiles, while delivering decompression throughput up to 16 GB/s on a 64-core ARM64 server. The codec supports three container formats: MIC1 for single frames, MIC2 for multi-frame sequences with optional temporal prediction, and MIC3 for tiled whole slide images with multi-resolution pyramid levels. A two-state FSE decoder breaks the serial dependency chain inherent in ANS decoding by running two independent state machines on alternating symbol positions, enabling instruction-level parallelism that yields 1.3–3.0× faster FSE decompression. Platform-specific Go assembly routines for histogram computation (amd64 and arm64) and YCoCg-R color transforms further reduce overhead. We evaluate MIC on eight clinical DICOM datasets spanning MR, CT, CR, X-ray, and mammography modalities, compare it against a 5/3 integer wavelet alternative, and benchmark it against High-Throughput JPEG 2000 (HTJ2K) using OpenJPH. Delta+RLE+FSE strictly dominates the wavelet pipeline on all tested modalities in both compression ratio and decompression speed. Compared to HTJ2K, MIC achieves comparable or better lossless compression ratios across all modalities and decompresses 1.3–1.5× faster on a single core with two-state FSE, scaling to 16 GB/s at 64 cores. MIC is implemented in Go with platform-specific assembly optimizations, JavaScript and WebAssembly browser decoders, and is released as open-source software.

1 Introduction

Medical imaging generates enormous volumes of data. A single digital breast tomosynthesis (DBT) study produces 69 or more frames of 2457×1890 pixels at 10-bit depth, totalling over 600 MB of raw pixel data. Whole slide pathology images routinely exceed 100 000×100 000 pixels. Efficient lossless compression is essential for archival storage, network transmission, and real-time rendering of these images in clinical picture archiving and communication systems (PACS) and diagnostic viewers.

The DICOM standard [**?**] supports several transfer syntaxes for lossless compression, including JPEG 2000 [**?**], JPEG-LS [**?**], and RLE Lossless. JPEG 2000 achieves strong compression ratios through its 5/3 integer wavelet transform but incurs high computational cost during decompression, which limits rendering throughput. JPEG-LS provides fast encoding and decoding but typically achieves lower compression ratios than JPEG 2000. DICOM RLE is simple but inefficient.

We introduce MIC, a codec that targets the intersection of high compression ratio and high decompression throughput. MIC exploits three properties of medical images:

1. **Spatial correlation**: Adjacent pixels are highly correlated, making a simple linear predictor highly effective.
2. **Residual sparsity**: After prediction, pixel residuals cluster tightly around zero with long runs of identical values.

*Innovaccer Inc. Email: kuldeep.singh@innovaccer.com. Code: https://github.com/pappuks/medical-image-codec.

3. **Moderate symbol cardinality**: Even 16-bit images typically produce fewer than 1 000 distinct residual values after delta coding.

The codec processes pixels through a three-stage pipeline: spatial delta encoding, run-length encoding, and FSE (ANS-based) entropy coding. Each stage is simple enough to decode in a single sequential pass with minimal branching, enabling decompression speeds that scale with memory bandwidth. A two-state FSE decoder further increases throughput by exploiting instruction-level parallelism (ILP) via dual independent ANS state machines. On a 64-core AWS Graviton3 server (c7g.metal), MIC decompresses mammography images at 16 GB/s—substantially faster than JPEG 2000 decoders.

Beyond single-frame greyscale, MIC supports:

- **Multi-frame images** (MIC2 container) with random-access frame indexing and optional inter-frame temporal prediction for video-like sequences.
- **Whole slide images** (MIC3 container) with tiled architecture, YCoCg-R reversible color transform, multi-resolution pyramid levels, and parallel tile compression.

The remainder of this paper is organized as follows. Section **??** surveys related work. Section **??** describes the compression pipeline. Section **??** details the MIC2 and MIC3 container formats. Section **??** covers implementation details and optimizations. Section **??** presents experimental results, including a comparison with HTJ2K (OpenJPH). Section **??** compares against a wavelet-based alternative. Section **??** discusses design tradeoffs and limitations. Section **??** concludes.

2 Related Work

2.1 Lossless Image Compression in DICOM

JPEG 2000 Part 1 [**?**] employs a 5/3 reversible integer wavelet transform with embedded block coding (EBCOT). It achieves excellent compression ratios but its bit-plane coding and context modeling impose significant computational overhead during decompression. High-Throughput JPEG 2000 (HTJ2K) [**?**] replaces EBCOT with a faster block coder, substantially improving decode speed while maintaining JPEG 2000 compatibility.

JPEG-LS [**?**] uses context-adaptive prediction (MED predictor) followed by Golomb-Rice coding. It provides fast encoding and decoding with moderate compression ratios. JPEG-LS is well-suited to smooth images but its prediction model is fixed and cannot adapt to the varying statistics of different medical modalities.

RLE Lossless, specified in the DICOM standard as Transfer Syntax 1.2.840.10008.1.2.5, uses byte-level PackBits run-length encoding. It is trivially simple but achieves poor compression ratios (typically $<1.5\times$) because it operates on bytes rather than 16-bit pixel values and lacks any decorrelation stage.

2.2 Entropy Coding

Traditional arithmetic coding achieves near-optimal compression but is inherently sequential: the data dependencies between encoder and decoder states resist parallelization. Mahapatra and Singh [**?**] addressed this with a parallel-pipelined FPGA implementation of multi-alphabet arithmetic coding, demonstrating that hardware acceleration can overcome the throughput limitations of sequential arithmetic coders. Asymmetric numeral systems (ANS) [**?**] take a different approach, generalizing arithmetic coding into a single-state machine that replaces the interval subdivision of arithmetic coding with integer state transitions. Finite State Entropy (FSE) [**?**] is a table-driven implementation of tANS (table-based ANS) that achieves near-optimal compression with $O(1)$ per-symbol encode and decode operations, without requiring the specialized hardware of [**?**]. FSE was popularized by its use in Zstandard [**?**] and is used in MIC for the final entropy coding stage.

Canonical Huffman coding [**?**] assigns codewords based on sorted code lengths, enabling compact codebook representation. MIC includes a canonical Huffman backend as an alternative to FSE; it produces slightly smaller output but decodes more slowly.

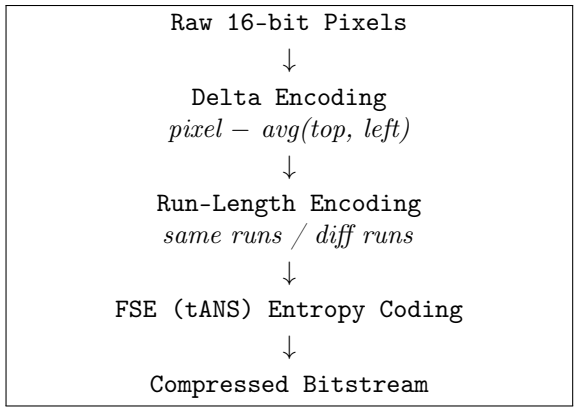

Figure 1: MIC compression pipeline for single-frame greyscale images.

2.3 Color Transforms for Pathology Imaging

The YCoCg-R transform [**?**] is a reversible integer approximation of the RGB-to-YCbCr color space conversion. It decorrelates RGB channels into luminance (Y) and chrominance (Co, Cg) with no loss of precision. YCoCg-R has been adopted in screen content coding extensions to H.265/HEVC and is used in MIC for whole slide image compression.

3 Compression Pipeline

MIC processes 16-bit greyscale pixels through three sequential stages. Figure **??** illustrates the dataflow.

3.1 Delta Encoding

The delta encoder computes the prediction residual for each pixel as:

$$r_{i,j} = x_{i,j} - \left\lfloor \frac{x_{i-1,j} + x_{i,j-1}}{2} \right\rfloor \tag{1}$$

where $x_{i,j}$ is the pixel at row i, column j. Boundary pixels use only the available neighbor (top for the first column, left for the first row) or the raw value for the top-left corner pixel.

For 16-bit images, residuals can span the range $[-65535, +65535]$. Rather than widening to 32 bits, MIC uses an *overflow delimiter* scheme derived from the image's effective bit depth $d = \lceil \log_2(\max(x) + 1) \rceil$:

- Residuals with $|r| \leq 2^{d-1} - 1$ are stored directly as uint16 values.
- Larger residuals are preceded by a delimiter value $2^d - 1$, followed by the raw pixel value.

This encoding adapts automatically to the actual bit depth of the image. For 8-bit images stored in 16-bit containers (common in MR), the threshold and delimiter operate within the 8-bit range, reducing the effective symbol alphabet and improving downstream entropy coding.

3.2 Run-Length Encoding

After delta coding, medical images produce long runs of identical residual values (especially zero) in smooth regions. MIC's RLE stage encodes the symbol stream as alternating *same runs* and *diff runs*:

- **Same run**: a count c followed by a single value repeated c times.
- **Diff run**: a count c followed by c distinct values.

A key design constraint is that the minimum run length is 3, which guarantees that the RLE output is *never larger* than the input. This eliminates the need for expansion detection or fallback paths.

The run headers use a midpoint protocol: counts $\leq$ `midCount` signal same runs; counts $>$ `midCount` signal diff runs with the actual count obtained by subtracting `midCount`.

3.3 FSE Entropy Coding

The final stage encodes the RLE output using table-based ANS (tANS), implemented as Finite State Entropy. MIC extends the standard FSE implementation to support up to 65 535 distinct symbols (versus the typical limit of 4 095), which is necessary for 16-bit medical images where delta residuals can span a wide range.

Encoding. The FSE encoder processes symbols in reverse order (last symbol first), building the compressed bitstream from the end. For each

symbol, the encoder transitions between states using a pre-computed symbol transition table (`symbolTT`), emitting bits that encode the state transitions.

Decoding. The decoder reads bits forward, using a decode table (`decTable`) indexed by the current state to produce the next symbol and compute the next state. Each decode step requires only a table lookup, a bit read, and an addition—no divisions or multiplications. A two-state variant (Section **??**) runs two independent decode state machines on alternating symbols to exploit instruction-level parallelism.

Adaptive table sizing. MIC automatically adjusts the FSE table log from the default of 11 to 12 when the symbol density is sufficiently high (>128 distinct symbols with >32 data points each). This provides 4–7% better compression ratios on CR and mammography images where the residual distribution has a wider spread.

Canonical Huffman alternative. MIC also supports a canonical Huffman entropy backend. The Huffman encoder adaptively selects the symbol set and limits tree depth to ≤14 bits for compact codebook representation. It typically produces 1–3% smaller output than FSE but decodes more slowly due to the variable-length code lookup.

4 Container Formats

4.1 MIC1: Single Frame

The simplest format. A compressed frame is prefixed with a header containing the image dimensions and maximum pixel value, followed by the compressed bitstream.

4.2 MIC2: Multi-Frame

MIC2 supports multi-frame DICOM images such as breast tomosynthesis (DBT). The format consists of:

- A 20-byte header: magic bytes (`"MIC2"`), width, height, frame count, and pipeline flags.
- A frame offset table ($N \times 8$ bytes): each entry stores the byte offset and length of one compressed frame, enabling $O(1)$ random access.
- Concatenated compressed frame blobs.

MIC2 supports two compression modes:

Independent mode. Each frame is compressed independently using the spatial Delta+RLE+FSE pipeline. This allows random access to any frame without decompressing preceding frames.

Temporal mode. Frame 0 is compressed spatially. Subsequent frames store inter-frame residuals: $r_t = \text{frame}_t - \text{frame}_{t-1}$. These signed residuals are mapped to unsigned values via ZigZag encoding ($z = 0$ for $r = 0$; $z = 2r - 1$ for $r > 0$; $z = 2|r|$ for $r < 0$), then compressed with RLE+FSE (no spatial delta, since temporal residuals lack spatial correlation).

4.3 MIC3: Whole Slide Imaging

MIC3 is a tiled container for whole slide pathology images. It extends the pipeline to handle RGB images via a reversible color transform.

YCoCg-R color transform. RGB pixels are converted to Y (luminance), Co (orange chrominance), and Cg (green chrominance) using the reversible YCoCg-R transform [**?**]:

$$\text{Co} = R - B \tag{2}$$

$$t = B + \lfloor \text{Co}/2 \rfloor \tag{3}$$

$$\text{Cg} = G - t \tag{4}$$

$$Y = t + \lfloor \text{Cg}/2 \rfloor \tag{5}$$

For 8-bit RGB inputs, Y lies in $[0, 255]$, while Co and Cg lie in $[-255, 255]$. The chrominance planes are ZigZag-encoded to $[0, 510]$ before compression. Each plane (Y, Co, Cg) is then compressed independently through the Delta+RLE+FSE pipeline.

Tiled architecture. Images are divided into tiles (default 256×256 pixels). Each tile is compressed independently, enabling:

- $O(1)$ random access to any tile via the tile offset table.
- Parallel compression and decompression using a goroutine worker pool.
- Per-tile optimization: constant tiles (e.g., white background) compress to 15–17 bytes.

Pyramid levels. MIC3 stores multiple resolution levels, each at half the dimensions of the previous. Downsampling uses a 2×2 box filter applied in RGB space before color transform. Level descriptors in the header record the dimensions and tile grid of each level, enabling a viewer to select the appropriate resolution for the current zoom level.

Constant-plane optimization. When all pixels in a plane have identical values, the plane is stored as a single constant (3 bytes: mode byte + uint16 value) rather than running the full compression pipeline. For white background tiles (common in pathology), all three planes are constant, yielding compression ratios exceeding $1\,900\times$.

5 Implementation and Optimizations

MIC is implemented in Go (module `mic`, Go 1.22+) with platform-specific assembly optimizations for amd64 and arm64. The implementation prioritizes decompression throughput, as the primary use case is real-time rendering of compressed images in viewers.

5.1 FSE Decode Loop Inlining

The innermost FSE decode loop in `decompress()` processes one symbol per iteration. State transitions are fully inlined with a local slice reference to the decode table, eliminating function call overhead and pointer indirections. This is the single most performance-critical code path in the codec.

5.2 Dual-Buffer Histogram

The symbol histogram function `countSimple()` uses two interleaved count arrays. Consecutive pixels are counted into alternating arrays, then merged. This reduces store-to-load forwarding stalls when consecutive pixels have similar values—a common pattern in medical images where adjacent pixels are highly correlated.

5.3 Branch-Free Delta Decompression

The delta decompressor uses four separate loops for corner, first-row, first-column, and interior pixels. The interior loop handles the vast majority of pixels ($> (w-1)(h-1)$ out of wh) without any per-pixel boundary checks, eliminating branch mispredictions.

5.4 RLE Fast Path

Same-value runs are the most common pattern after delta coding. The RLE decoder fast-paths same runs to return the cached value without touching the input slice, requiring only a counter decrement per output symbol.

5.5 Dynamic Table Sizing

FSE encode and decode tables are sized to the actual symbol range rather than the theoretical maximum of 65 536. For 8-bit images, this reduces the working set from 512 KB to approximately 2 KB, substantially improving cache utilization.

5.6 Two-State FSE Decoder

The standard FSE decode loop has a serial carried-dependency chain: each state transition depends on the result of the previous one. With a table-lookup latency of approximately 4 cycles on modern out-of-order CPUs, the loop is limited to roughly one symbol per 4 cycles regardless of execution unit availability.

MIC's two-state FSE decoder breaks this serial dependency by running two independent state machines (`stateA` and `stateB`) on alternating

Table 1: Two-state FSE decompression speedup (Apple M2 Max, 12 cores, full Delta+RLE+FSE pipeline, 200 iterations).

Image	1-state (MB/s)	2-state (MB/s)	Speedup
MR (256×256)	1 404	2 285	1.63×
CT (512×512)	1 556	2 029	1.30×
CR (2140×1760)	3 778	5 323	1.41×
XR (2048×2577)	3 890	5 787	1.49×
MG1 (2457×1996)	3 722	5 148	1.38×
MG2 (2457×1996)	3 637	4 751	1.31×
MG3 (4774×3064)	1 917	5 705	**2.98**×
MG4 (4096×3328)	4 230	6 001	1.42×

symbol positions:

$$\text{symbol}[0] \leftarrow \text{stateA}, \quad \text{symbol}[2] \leftarrow \text{stateA}, \quad \ldots$$
$$\text{symbol}[1] \leftarrow \text{stateB}, \quad \text{symbol}[3] \leftarrow \text{stateB}, \quad \ldots$$

Since the two chains are completely independent, the CPU's out-of-order engine can issue both table lookups simultaneously, filling the 4-cycle latency slots with useful work from the other chain.

The encoder processes symbols in reverse (as required by ANS), encoding even-indexed symbols with state A and odd-indexed symbols with state B. The compressed stream is prefixed with a two-byte magic sequence (`0xFF`, `0x02`) followed by a 4-byte symbol count, enabling the auto-detect dispatcher (`FSEDecompressU16Auto`) to route streams to the correct decoder transparently. The magic byte `0xFF` cannot appear as a valid single-state FSE header (it would imply `tableLog` $= 20 > 16$), so backward compatibility is preserved without ambiguity.

Table **??** reports the decompression speedup of two-state FSE over single-state FSE on a 12-core Apple M2 Max.

Two-state FSE delivers 1.3–3.0× faster decompression across all modalities. The technique is implemented in pure Go and benefits every platform (amd64, arm64, WebAssembly) without any assembly code.

5.7 Go Assembly Optimizations

MIC includes platform-specific Go assembly routines for amd64 (Intel/AMD) and arm64 (Apple Silicon, AWS Graviton), with pure-Go fallbacks for all other architectures.

Interleaved histogram. The symbol frequency histogram (`countSimpleU16Asm`) distributes even-indexed pixels into one count array and odd-indexed pixels into a second, using a 4-way unrolled loop. This avoids store-to-load forwarding stalls when consecutive pixels have identical values—a common pattern in medical images after delta coding. The two arrays are merged in a single backward pass after the loop. Both amd64 (using `ADDL`/`MOVWQZX` instructions) and arm64 (using `MOVWU`/`LSL`/`ADD`) implement the same algorithm in Go Plan 9 assembly.

YCoCg-R color transform. The RGB→YCoCg-R forward and inverse transforms are implemented in assembly with explicit register allocation, eliminating Go compiler overhead for the per-pixel arithmetic. On amd64, a one-time `CPUID` probe at initialization detects SSSE3 and AVX2 support; on arm64, NEON is always available. The current scalar-in-assembly implementations establish the dispatch scaffolding for future SIMD-vectorized (8-pixel-wide) paths.

Platform portability. A build-tagged fallback file (`asm_generic.go`) provides identical pure-Go implementations for non-amd64/arm64 targets (WebAssembly, RISC-V, etc.), ensuring the codec compiles and runs correctly on all Go-supported platforms.

5.8 Browser Decoders

MIC includes both a pure JavaScript ES module decoder (∼20 KB, zero dependencies) and a Go WebAssembly build. The JavaScript decoder achieves 10–30 M pixels/s in V8, sufficient for interactive viewing. Both decoders support MIC1, MIC2, and MIC3 formats, including multi-frame movie playback and RGB WSI tile rendering with window/level controls.

Table 2: Test dataset: clinical DICOM images.

Image	Modality	Dimensions	Raw Size
MR	Brain MRI	256×256	0.13 MB
CT	CT scan	512×512	0.50 MB
CR	Comp. radiography	2140×1760	7.18 MB
XR	X-ray	2048×2577	10.1 MB
MG1	Mammography	2457×1996	9.35 MB
MG2	Mammography	2457×1996	9.35 MB
MG3	Mammography	4774×3064	27.3 MB
MG4	Mammography	4096×3328	26.0 MB

Table 3: Lossless compression ratios (Delta+RLE+FSE).

Image	Raw Size	Compressed	Ratio
MR	0.13 MB	0.053 MB	2.35×
CT	0.50 MB	0.22 MB	2.24×
CR	7.18 MB	1.98 MB	3.63×
XR	10.1 MB	5.79 MB	1.74×
MG1	9.35 MB	1.09 MB	8.57×
MG2	9.35 MB	1.09 MB	8.55×
MG3	27.3 MB	12.2 MB	2.24×
MG4	26.0 MB	7.49 MB	3.47×

6 Experimental Results

6.1 Test Dataset

We evaluate MIC on eight clinical DICOM images spanning five modalities (Table **??**). All images are 16-bit greyscale. Effective bit depth ranges from 8–12 bits (MR) to the full 16-bit range (CT).

6.2 Compression Ratios

Table **??** reports lossless compression ratios for the Delta+RLE+FSE pipeline.

Mammography images achieve the highest ratios (up to 8.57×) due to large smooth breast tissue regions that produce near-zero residuals after delta coding. CT images with full 16-bit dynamic range achieve lower ratios (2.24×) because the wider value range increases residual entropy. X-ray images achieve the lowest ratios (1.74×) due to high-frequency noise.

Adaptive tableLog improvement. Automatically increasing the FSE table log from 11 to 12 for images with high symbol density yields measurable gains (Table **??**).

Table 4: Effect of adaptive tableLog (11→12).

Image	Before	After	Gain
CR	3.47×	3.63×	+4.4%
MG1	7.99×	8.57×	+7.1%
MG2	7.98×	8.55×	+7.1%

6.3 Decompression Throughput

Table **??** reports decompression throughput across four hardware platforms. The primary benchmark metric is MB/s of raw (uncompressed) pixel data produced. Table **??** expresses the same results as frames per second (FPS) on the AWS c7g.metal platform.

Key observations:

- **Peak throughput of 16.4 GB/s** is achieved on MG1 (mammography) on the 64-core ARM64 platform, approaching DRAM bandwidth limits.
- Throughput scales roughly linearly with core count: the 32-core ARM64 instance achieves approximately half the throughput of the 64-core instance.
- ARM64 platforms outperform x86 at equivalent core counts, likely due to wider memory buses on AWS Graviton3 instances.
- Larger images achieve higher throughput due to better amortization of fixed overhead (header parsing, table construction).
- RAM bandwidth is the primary bottleneck, not CPU speed. DDR5 platforms consistently outperform DDR4 platforms.

6.4 Multi-Frame Results

Table **??** reports compression results for a 69-frame breast tomosynthesis study (2457×1890, 10-bit depth).

Independent mode slightly outperforms temporal mode on this dataset because mammography images have strong spatial correlation (the spatial predictor is very effective) but only moderate inter-frame correlation (each slice images a different plane of breast tissue). Temporal mode

Table 5: Decompression throughput (MB/s of raw pixel data) across hardware platforms.

Platform	MR	CT	CR	XR	MG1	MG2	MG3	MG4
AWS c7g.metal (ARM64, 64c)	2 282	4 433	8 527	9 411	**16 387**	16 023	8 044	15 213
AWS c7g.8xl (ARM64, 32c)	1 524	2 186	4 290	4 562	8 901	7 879	4 455	7 132
AWS c7i.8xl (x86, 32c, Xeon 8488C)	1 142	1 208	3 172	3 269	5 220	5 124	3 468	4 964
Mac Studio (M2 Max, 12c)	1 054	1 121	2 089	2 101	3 666	3 659	2 239	3 188

Table 6: Decompression in frames per second (FPS) on AWS c7g.metal.

Image	FPS	MB/s
MR (256×256)	17 411	2 282
CT (512×512)	8 455	4 433
CR (2140×1760)	1 132	8 527
XR (2048×2577)	892	9 411
MG1 (2457×1996)	1 671	16 387
MG2 (2457×1996)	1 634	16 023
MG3 (4774×3064)	281	8 044
MG4 (4096×3328)	558	15 213

Table 7: Multi-frame compression (69-frame DBT, 614 MB raw).

Mode	Compressed	Ratio
Independent (spatial)	46.1 MB	13.3×
Temporal (inter-frame)	47.5 MB	12.9×

may be advantageous for sequences with higher inter-frame correlation, such as cardiac cine MRI or fluoroscopy.

6.5 WSI Compression Results

Table **??** reports compression results for whole slide image tiles.

The YCoCg-R color transform is particularly effective for pathology images. H&E-stained tissue produces smooth color gradients in the luminance and chrominance planes, yielding strong spatial prediction. Background tiles (slide glass regions) are nearly constant and compress to 15–17 bytes total via the constant-plane optimization.

6.6 Comparison with HTJ2K

We compare MIC against lossless HTJ2K using OpenJPH [**?**] v0.15 (`ojph_compress -reversible true` / `ojph_expand`), the leading open-source HTJ2K implementation. Measurements are single-threaded on an Apple M2 Max workstation. MIC uses the two-state FSE decoder (Section **??**). MIC timings are in-process (pure library calls); HTJ2K timings include subprocess launch, PGM file I/O, and JPEG 2000 codestream I/O overhead (approximately 6 ms per invocation).

Table **??** reports lossless compression ratios and single-threaded decompression throughput. Decompression throughput is the minimum of five runs (standard benchmarking practice to eliminate OS scheduling noise).

Table 8: WSI tile compression ratios (MIC3, YCoCg-R + Delta+RLE+FSE).

Tile Type	Ratio	Notes
White background	1 946×	Near-zero entropy
Dense tissue (H&E)	4.4×	Smooth staining gradients
Gradient	5.4×	Excellent spatial correlation
Mixed slide	4–8×	Weighted avg. of tile types

Compression ratio. Ratios are within 4% across all modalities. MIC achieves better ratios on CT (+27%), XR (+4%), and MG1/MG2 (+4%). HTJ2K achieves slightly better ratios on MR (+1%), CR (+4%), and MG4 (+1%). The CT advantage of MIC stems from better handling of the full 16-bit dynamic range: the overflow delimiter scheme adapts to the actual bit depth, keeping the symbol alphabet small, while HTJ2K's block coder operates on fixed 16-bit coefficients.

Decompression throughput. For images where the subprocess overhead is negligible

Table 9: Lossless compression comparison: MIC (Delta+RLE+FSE, two-state decoder) vs. HTJ2K (OpenJPH [?], `-reversible true`). Single-threaded, Apple M2 Max. MIC timings are in-process; HTJ2K timings include subprocess and I/O overhead (≈6 ms per call).

Image	Raw (MB)	MIC ratio	HTJ2K ratio	MIC (MB/s)	HTJ2K (MB/s)	MIC speedup
MR	0.13	2.35×	**2.38**×	133	20 †	—
CT	0.50	**2.24**×	1.77×	164	82 †	—
CR	7.18	3.63×	**3.77**×	**287**	215	1.33×
XR	10.1	**1.74**×	1.67×	**297**	205	1.45×
MG1	9.35	**8.57**×	8.25×	**471**	338	1.39×
MG2	9.35	**8.55**×	8.24×	**471**	338	1.39×
MG3	27.3	**2.24**×	2.22×	**297**	225	1.32×
MG4	26.0	3.47×	**3.51**×	**399**	307	1.30×

† Process startup dominates; not representative of HTJ2K library performance.

(≥7 MB: CR, XR, MG1–MG4), MIC with the two-state FSE decoder decompresses 1.3–1.5× faster than HTJ2K on a single thread (up from 1.1–1.2× with the single-state decoder). The two-state decoder's ILP gains (Section **??**) directly translate to improved throughput in the HTJ2K comparison: CR improves from 242 to 287 MB/s, MG1 from 428 to 471 MB/s, and MG4 from 353 to 399 MB/s. At 64 cores (AWS c7g.metal), MIC achieves up to 16 GB/s (Table **??**), whereas the OpenJPH CLI is limited to a single-process throughput of 2–4 GB/s. The difference arises from architecture: MIC is a library with zero subprocess overhead and decompresses independent frames in parallel goroutines, while OpenJPH's command-line tool processes one file at a time.

Summary. MIC achieves comparable or better lossless compression ratios to HTJ2K on the tested medical image modalities, while providing 1.3–1.5× higher single-threaded decompression throughput via the two-state FSE decoder, scaling to 16 GB/s at 64 cores. HTJ2K's strengths lie in JPEG 2000 ecosystem compatibility and support for progressive/region-of-interest decoding, which MIC does not currently provide for greyscale images.

7 Comparison with Wavelet-Based Compression

To evaluate whether a more sophisticated decorrelation stage could improve MIC, we implemented a Le Gall 5/3 integer wavelet transform [?]—the same transform used in JPEG 2000 lossless mode—as an alternative to delta encoding.

7.1 Wavelet Implementation

The 5/3 lifting scheme applies predict and update steps:

$$d[i] = x[2i{+}1] - \left\lfloor \frac{x[2i] + x[2i{+}2]}{2} \right\rfloor \tag{6}$$

$$s[i] = x[2i] + \left\lfloor \frac{d[i{-}1] + d[i] + 2}{4} \right\rfloor \tag{7}$$

The 2D transform applies 1D transforms to all rows, then all columns. Wavelet coefficients are ZigZag-encoded to unsigned values, with an overflow escape for coefficients exceeding the uint16 range.

We evaluated two wavelet pipelines:

- Wavelet+FSE: wavelet transform → FSE (no RLE).
- Wavelet+RLE+FSE: wavelet transform → RLE → FSE.

7.2 Compression Ratio Comparison

Table **??** compares compression ratios. Delta+RLE+FSE achieves the best ratio

Table 10: Compression ratio: Delta+RLE+FSE vs. wavelet pipelines.

Image	Delta+ RLE+FSE	Wavelet +FSE	Wavelet+ RLE+FSE
MR	**2.35**×	2.09×	2.09×
CT	**2.24**×	1.48×	1.48×
CR	**3.63**×	2.59×	2.59×
XR	**1.74**×	1.53×	1.53×
MG1	**8.57**×	4.91×	7.28×
MG2	**8.55**×	4.90×	7.27×
MG3	**2.24**×	1.90×	1.93×
MG4	**3.47**×	2.63×	3.11×

Table 11: Decompression speed (MB/s): Delta+RLE+FSE vs. wavelet.

Image	Delta+ RLE+FSE	Wavelet +FSE	Wavelet+ RLE+FSE
MR	116	**146**	122
CT	165	**168**	142
CR	**543**	418	371
XR	**605**	576	486
MG1	**1 530**	592	680
MG2	**1 493**	618	644
MG3	**606**	387	352
MG4	**1 054**	480	579

on *every* modality.

7.3 Decompression Speed Comparison

Table **??** compares single-threaded decompression speed. Delta+RLE+FSE is faster on all large images (≥2K pixels); the wavelet pipeline wins only on small images (MR, CT) where the overhead of the wavelet inverse is amortized.

7.4 Analysis

Delta encoding outperforms the wavelet transform for lossless medical image compression for three reasons:

1. **The linear predictor is near-optimal.** Medical images have smooth intensity gradients and large homogeneous regions. The average-of-neighbors predictor produces residuals tightly clustered around zero. RLE then compresses long runs of identical residuals directly.
2. **The wavelet update step hurts.** The 5/3 update step reintroduces correlation in the low-pass band to preserve the signal mean (needed for perfect reconstruction). For images where delta residuals are already near-zero, this provides no net benefit. For full-range CT images, the update step forces escape encoding when low-pass coefficients overflow uint16, inflating the compressed stream.
3. **Delta decompression uses half the memory bandwidth.** The wavelet inverse requires two full passes over the pixel buffer (horizontal and vertical), each touching every element twice. It also operates on int32 values (4 bytes/sample) versus uint16 (2 bytes/sample) for delta. For large mammography images at high parallelism, this doubles the memory bandwidth requirement, which is the binding constraint.

The wavelet transform remains valuable for two use cases outside MIC's current scope: *lossy compression* (where wavelet coefficients can be quantized by subband) and *progressive/multi-resolution delivery* (where the LL subband is a valid downscaled image at each decomposition level).

8 Discussion

Design philosophy. MIC deliberately avoids complex prediction models (context modeling, Markov predictors) in favor of a simple three-stage pipeline where each stage has $O(n)$ complexity and minimal branching. This design sacrifices a small amount of compression ratio (compared to JPEG 2000) in exchange for dramatically higher decompression throughput. For clinical workflows where images are compressed once and decompressed many times (archival → viewing), this tradeoff is strongly favorable.

Bit-depth adaptivity. A notable feature of MIC is its automatic adaptation to the effective bit depth of the image. All thresholds and delim-

iters are derived from the actual maximum pixel value via $\lceil \log_2(\max + 1) \rceil$, eliminating the need for separate 8-bit and 16-bit code paths. This simplifies the implementation and ensures optimal performance across the full range of DICOM bit depths (8–16 bits).

Limitations. MIC does not support lossy compression, progressive decoding, or region-of-interest coding for greyscale images. These capabilities are available in JPEG 2000 and may be important for bandwidth-limited telemedicine applications. The wavelet transform, already implemented and benchmarked, provides a natural foundation for a future lossy mode.

For WSI images, MIC3's tiled architecture provides tile-level random access and pyramid-level multi-resolution support, addressing the region-of-interest and progressive delivery requirements specific to digital pathology.

Comparison with HTJ2K. High-Throughput JPEG 2000 (HTJ2K) [**?**] is the closest point of comparison for MIC. Section **??** presents a quantitative comparison against OpenJPH [**?**], the leading open-source HTJ2K implementation. MIC achieves comparable or better lossless compression ratios and, with the two-state FSE decoder (Section **??**), decompresses 1.3–1.5× faster on a single thread (Apple M2 Max)—an improvement over the 1.1–1.2× advantage of the single-state decoder. At 64 cores, MIC's advantage grows substantially: the simple per-frame parallelism of the Delta+RLE+FSE pipeline scales to 16 GB/s, while the OpenJPH command-line tool is limited by its single-process architecture. HTJ2K's key advantage is JPEG 2000 ecosystem compatibility and support for progressive and region-of-interest decoding, which MIC does not currently provide for greyscale images.

9 Conclusion

We have presented MIC, a lossless medical image codec that achieves compression ratios of 1.7×–8.6× on greyscale DICOM images and 3–5× on RGB pathology images, with decompression throughput up to 16 GB/s. The codec's three-stage pipeline (delta prediction, run-length encoding, FSE entropy coding) is designed for simplicity and speed: each stage processes the data in a single pass with minimal branching, enabling decompression performance that scales with memory bandwidth.

A two-state FSE decoder that runs two independent ANS state machines on alternating symbols delivers 1.3–3.0× faster FSE decompression by exploiting instruction-level parallelism, without requiring any platform-specific assembly. Platform-specific Go assembly routines for histogram computation and YCoCg-R color transforms on amd64 and arm64 further reduce overhead, with pure-Go fallbacks ensuring portability to all architectures.

MIC supports three container formats for different clinical use cases: single-frame (MIC1), multi-frame with temporal prediction (MIC2), and tiled whole slide imaging with pyramid levels (MIC3). The implementation includes browser-based JavaScript and WebAssembly decoders for web viewing applications.

Our evaluation demonstrates that the simple delta+RLE+FSE pipeline strictly dominates a wavelet-based alternative on all tested DICOM modalities, in both compression ratio and decompression speed. This result suggests that for lossless medical image compression, simple linear prediction combined with fast entropy coding is more effective than the sophisticated wavelet transforms used in JPEG 2000. A quantitative comparison against HTJ2K (OpenJPH) shows that MIC achieves comparable or better lossless compression ratios across all modalities and, with the two-state FSE decoder, decompresses 1.3–1.5× faster on a single core (Apple M2 Max), scaling to 16 GB/s on a 64-core ARM64 server—substantially higher than the throughput achievable by a single-process HTJ2K implementation.

MIC is open-source and available at https://github.com/pappuks/medical-image-codec.

Availability. Source code, test data, and benchmark scripts are available under an open-source license at the URL above.

References

[1] NEMA, "Digital Imaging and Communications in Medicine (DICOM)," https://www.dicomstandard.org/, 2024.

[2] D. S. Taubman and M. W. Marcellin, "JPEG2000: Image Compression Fundamentals, Standards and Practice," Springer, 2002.

[3] D. S. Taubman, "High throughput JPEG 2000 (HTJ2K): Algorithm, performance evaluation, and potential," *Proc. SPIE*, vol. 11137, 2019.

[4] M. J. Weinberger, G. Seroussi, and G. Sapiro, "The LOCO-I lossless image compression algorithm: Principles and standardization into JPEG-LS," *IEEE Trans. Image Process.*, vol. 9, no. 8, pp. 1309–1324, 2000.

[5] S. Mahapatra and K. Singh, "An FPGA-based implementation of multi-alphabet arithmetic coding," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 54, no. 8, pp. 1678–1686, Aug. 2007.

[6] J. Duda, "Asymmetric numeral systems," *arXiv preprint arXiv:0902.0271*, 2009.

[7] Y. Collet, "Finite State Entropy—A new breed of entropy coder," https://github.com/Cyan4973/FiniteStateEntropy, 2013.

[8] Y. Collet and M. Kucherawy, "Zstandard Compression and the 'application/zstd' Media Type," RFC 8878, Internet Engineering Task Force, 2021.

[9] E. S. Schwartz and B. Kallick, "Generating a canonical prefix encoding," *Commun. ACM*, vol. 7, no. 3, pp. 166–169, 1964.

[10] H. S. Malvar and G. J. Sullivan, "YCoCg-R: A color space with RGB reversibility and low dynamic range," *Joint Video Team (JVT) Doc. JVT-I014*, 2003.

[11] D. Le Gall and A. Tabatabai, "Sub-band coding of digital images using symmetric short kernel filters and arithmetic coding techniques," *Proc. IEEE ICASSP*, pp. 761–764, 1988.

[12] A. N. Aous, "OpenJPH: An open-source implementation of High-Throughput JPEG 2000," https://github.com/aous72/OpenJPH, 2024.